INTERFACEGUARD

Object-Oriented Interface Erosion Audit

Repository: **codeshovel** • URL: https://github.com/ataraxie/codeshovel • Generated: 2026-10-18 15:02 UTC

1. Executive Summary & Project Erosion Index

Project Erosion Index	**53.8 / 100**	**Status:**	**Eroded**
Analyzed Commits:	6	**Tracked Interfaces:**	3
ISP Violations:	2	**Dependency Cycles:**	2

2. Interface Inventory & CK Metrics

CK metrics (CBO, LCOM, RFC, WMC, DIT) are aggregated across implementing classes. NOC represents the number of concrete implementing classes.

Interface	Pkg	Health	Severity	Methods	CBO	LCOM	WMC	NOC
Yfunction	`com.felixgru`	45.9	HIGH	17	7.5	0.60	23.0	2
Yparser	`com.felixgru`	57.8	HIGH	11	13.0	0.76	15.5	2
RepositoryService	`com.felixgru`	57.6	HIGH	13	13.0	0.72	16.0	1

3. Interface Segregation Principle (ISP) Violations

Rule / Indicator	Class	Method	Evidence
Empty Implementation Method	`AbstractParser`	`getScopeSimilarity`	Method body is completely empty, suggesting the class was forced to implement an unused method.
Empty Implementation Method	`AbstractParser`	`getAcceptedFileExtension`	Method body is completely empty, suggesting the class was forced to implement an unused method.

4. Design Smells & Dependency Cycles

Category	Rule Name	Severity	Target	Details
Code Smell	**Fat Interface**	CRITICAL	`C:/Users/Windows/App`	Interface 'Yfunction' declares 17 methods with low cohesive parameter sharing (0%).
Code Smell	**God Class**	CRITICAL	`AbstractFunction`	Class 'AbstractFunction' exhibits high complexity, extensive coupling, and low cohesion.

Code Smell	**Large Class**	WARNING	`AbstractFunction`	Implementing class 'AbstractFunction' declares 31 methods.
Code Smell	**Feature Envy**	WARNING	`AbstractFunction`	Method 'getIdParameterString' in 'AbstractFunction' delegates heavily to external methods: getParameters, add, toString.
Code Smell	**Feature Envy**	WARNING	`AbstractFunction`	Method 'getSourceFragment' in 'AbstractFunction' delegates heavily to external methods: getNameLineNumber, getEndLineNumber, getSourceFileContent.
Code Smell	**God Class**	CRITICAL	`JavaFunction`	Class 'JavaFunction' exhibits high complexity, extensive coupling, and low cohesion.
Code Smell	**Large Class**	WARNING	`JavaFunction`	Implementing class 'JavaFunction' declares 15 methods.
Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'createParameterMetadataMap' in 'JavaFunction' delegates heavily to external methods: createParameterModifiersString, put, createParameterAnnotationsString.
Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'createParameterModifiersString' in 'JavaFunction' delegates heavily to external methods: getModifiers, add, asString.
Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'createParameterAnnotationsString' in 'JavaFunction' delegates heavily to external methods: getAnnotations, add, toString.
Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'getInitialParameters' in 'JavaFunction' delegates heavily to external methods: getParameters, getNameAsString, getTypeAsString.
Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'getInitialBody' in 'JavaFunction' delegates heavily to external methods: isPresent, getBody, toString.
Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'getInitialBeginLine' in 'JavaFunction' delegates heavily to external methods: isPresent, getBegin, getName.

Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'getInitialEndLine' in 'JavaFunction' delegates heavily to external methods: isPresent, getEnd, get.
Code Smell	**Feature Envy**	WARNING	`JavaFunction`	Method 'getInitialParentName' in 'JavaFunction' delegates heavily to external methods: isPresent, getParentNode, get.
Cycle	**Circular Dependency**	CRITICAL	`Cycle`	Cycle path: Ycommit -> Yparser -> Ycommit
Cycle	**Circular Dependency**	CRITICAL	`Cycle`	Cycle path: Ycommit -> Yparser -> Ycommit

5. Historical Evolution & Statistical Change Points

No abrupt statistical change points detected in the analyzed commit window.

6. AI-Assisted Architecture & Refactoring Recommendations

Diagnosis (deterministic_expert_engine):

Interface 'Yfunction' exhibits design erosion symptoms. Implementing classes are forced to provide 2 empty stubs or UnsupportedOperationException stubs, directly violating the Interface Segregation Principle. The interface has grown to 17 methods, accumulating unrelated operational concerns.

Refactoring Plan & Responsibility Separation:

Apply the 'Extract Interface' refactoring pattern: - Split 'Yfunction' into focused role-based interfaces with <= 3 methods each. - Have implementing classes implement only the specific interfaces they actually fulfill.

Risks & Trade-offs: Refactoring interface contracts introduces temporary breaking changes for external callers. Deprecate bloated methods first, provide default adaptor implementations, and migrate client call sites incrementally.

Diagnosis (deterministic_expert_engine):

Interface 'Yparser' exhibits design erosion symptoms. Implementing classes are forced to provide 2 empty stubs or UnsupportedOperationException stubs, directly violating the Interface Segregation Principle. The interface has grown to 11 methods, accumulating unrelated operational concerns.

Refactoring Plan & Responsibility Separation:

Apply the 'Extract Interface' refactoring pattern: - Split 'Yparser' into focused role-based interfaces with <= 3 methods each. - Have implementing classes implement only the specific interfaces they actually fulfill.

Risks & Trade-offs: Refactoring interface contracts introduces temporary breaking changes for external callers. Deprecate bloated methods first, provide default adaptor implementations, and migrate client call sites incrementally.

Diagnosis (deterministic_expert_engine):

Interface 'RepositoryService' exhibits design erosion symptoms. Implementing classes are forced to provide 2 empty stubs or UnsupportedOperationException stubs, directly violating the Interface Segregation Principle. The interface has grown to 13 methods, accumulating unrelated operational concerns.

Refactoring Plan & Responsibility Separation:

Apply the 'Extract Interface' refactoring pattern: - Split 'RepositoryService' into focused role-based interfaces with <= 3 methods each. - Have implementing classes implement only the specific interfaces they actually fulfill.

Risks & Trade-offs: Refactoring interface contracts introduces temporary breaking changes for external callers. Deprecate bloated methods first, provide default adaptor implementations, and migrate client call sites incrementally.

7. Methodology & Known Limitations

Methodology: InterfaceGuard analyzes Java source code across Git history using JavaParser AST parsing and PyDriller. Metric extraction includes Chidamber & Kemerer (CK) metrics aggregated over implementing classes, interface cohesion metrics, ISP violation heuristics (empty/no-op/exception/placeholder stubs), and NetworkX cycle analysis.

Limitations: Dynamic runtime reflection and bytecode manipulation are out of scope. Statistical change-point detection highlights sudden score deteriorations but does not prove intent. All recommendations should be reviewed by engineering teams.